

PAC (Phase-Amplitude Coupling) 机制解析与代码修复验证记录 (0502)

本篇文档记录了项目中关于相位-振幅耦合 (PAC) 计算模块的核心讨论、原版 (v7) 存在的数学偏差及其修复逻辑、详尽的质检验证过程，以及对新代码 `compute_pac_metrics_fixed.py` 的逐块拆解和直理解。

1. PAC 到底在测什么？(从傅里叶到瞬时特征)

标准傅里叶变换 (FFT) 能够拆解出信号的频率成分，但它给出的是"全局平均图景"，缺乏时间维度的动态协同刻画。在睡眠网络中，我们真正关心的是：**皮层慢波 (SO) 的相位（当前周期进行到了哪里）是否在指挥、控制着丘脑纺锤波 (Spindle) 振幅的爆发？** 这个现象叫做 PAC。

我们的测量分为 5 个核心步骤：

- 分离组分**：用带通滤波器分别提取皮层的低频慢波（0.5-1.5Hz）和丘脑的高频纺锤波（10-14Hz）。
- 提取瞬时特征 (Hilbert 变换)**：
 - 求慢波的**瞬时相位**（类似过山车开到了哪：波峰是0度，波谷是±180度）。
 - 求纺锤波的**瞬时振幅**（包络能量，即此时游客的尖叫声有多大）。
- 调制指数 (MI, Modulation Index) - 真的有耦合吗？** 把慢波周期切分成多个箱子。测的不是"spindle何时爆发"，而是"spindle振幅在SO相位bins上的分布有多不均匀"。如果纺锤波振幅在每个相位bin都差不多，分布是均匀的，香农熵最大， $MI \approx 0$ ；如果纺锤波振幅集中在某几个相位bin，分布偏离uniform，信息熵下降， $MI > 0$ ，证明存在耦合。
- 偏好相位 (Preferred Phase) - 谁在什么位置发号施令？** 使用平均向量长度 (Mean Vector Length)。好比一场复平面上的拔河比赛：以慢波的相位为方向，以纺锤波的振幅为力气。加总求平均后的最终箭头指向的角度，就是**高频最爱爆发的低频位置**（理想生理状态下应在 UP-state，即波峰 0度 附近）。
- 时滞推断 (Lag) - 因果关系** 使用交叉相关 (Cross-correlation)。把慢波和纺锤波的两根线进行错位滑移，找到重合度（乘积和）最高时的滑移耗时，借此判断究竟是皮层慢波"超前"引发了纺锤波，还是相反。

2. 老版代码 (V7) 遇到了什么危机？—— "哈哈镜里的扭曲"

在 V7 原版代码中，计算慢波相位使用了标准组合拳：`带通滤波 + Hilbert变换`。但这种高等数学工具具有一个隐藏前提（Bedrosian定理）：它假设输入的波是平滑的正弦波 $\sin(x)$ 。

然而，我们用模型跑出来的皮层信号实际上是**带有尖锐突起和长距离平地的脉冲波**。把它丢进滤波器后会出现两个核心bug：

Bug 1：波峰滞后（Phase shift artifact）

滤波器(bandpass)就像一个"模糊镜头"——它把瞬间的尖锐脉冲拍成了一团模糊的光斑。因为脉冲是**先冲上去再慢慢衰减**（像音叉敲一下），不是对称的，所以模糊镜头拍出的光斑**重心(亮度最高的地方)落在脉冲后面50ms**——不是因为时间倒流了，是因为脉冲的"质量"大部分集中在峰值之后。

结果：算出的波形最高点比真实的电活动**推迟了约 50 毫秒**，导致 Hilbert phase=0 不再对应真实 UP peak。

Bug 2：相位采样不均（Sampling imbalance）

皮层信号在 DOWN-state 平地上**停留时间比 UP-state 尖峰长得多（约8倍）**——这是真实的物理现象，不是计算失真。

- DOWN state 持续 ~1100ms（长平地）
- UP-to-DOWN transition 仅 ~150ms（尖锐脉冲）

把信号丢进bandpass + Hilbert后，Hilbert phase 在 DOWN state 附近"逗留时间长"，自然采样更多。结果18个相位箱子里 ± 180 度附近的箱子被装入**超过3.3倍的样本数**。这种 unbalanced sampling 让平均向量计算偏向DOWN trough。

后果

旧版就像一面"哈哈镜"，把原本瞄准波峰 (0度) 的生理耦合，错误算成了 159度 或偏离 -12.2度，干扰了优化器的演化方向。

3. 修复方案：Cycle-by-cycle Phase (朴素但致命的精度)

在修复版 `compute_pac_metrics_fixed.py` 中，我们舍弃了这套"滤镜"，采用了**逐周期分析法 (Cycle-by-cycle analysis, Cole & Voytek 2017)**：

1. **眼见为实的找山顶**：直接在原始电信号波形上使用 `find_peaks` 找极值锋芒，霸气声明：真实尖峰的所在地，就是绝对的 0度！
2. **时间戳线性插值**：第一座山顶是 0，第二座也是 0。期间不管怎么洼陷，一律按时间距离"均分切割"出 360 度。

这种方法完全免除了滤波带来的滞后和采样不均，强行纠正了评价机制的视线，让它重新盯住了最真实的"UP 爆发同步"。

4. 严苛的出厂质检：验证脚本的 9 大关卡

有了新的 PAC 尺子，我们在 `validate_compute_pac_metrics_fixed.py` 中构建了一个极为严酷的虚拟出厂质检车间，捏造出了含"标准答案"的测试假信号。

测试脚本一次性通过了所有 9 大挑战 (`9 / 9 validations passed`)，证明了算法坚不可摧：

- **V1 (基准能力)**：验证能完美夺回锁定在 0 度的基准合成数据。
- **V2 (波形抗干扰)**：无论把慢波捏造成圆浪、宽砖块、还是极长极细的"尖锋刺针"（老版的死穴），新算法一律稳定输出 0.0 度的标准答案，完全不被波形扭曲。
- **V3 (防假阳性)**：随机撒网纺锤波，新尺子能冷酷证实 MI 极低。
- **V4 (三大生理状态)**：完美区分在波峰、波谷及上升沿发生的三种截然不同的耦合模式。
- **V5 (噪声鲁棒性测试)**：从无噪声逐步加大白噪音 σ （最高到 20），相位都能 **recover** 到 $\pm 10^\circ$ 内。
- **V6 (老旧正面对决)**：在相同的尖刺数据下，证实老 Hilbert 版偏离去了 -12.2 度，而新版测出 -0.0 度。
- **V7 (相位采样均匀性)**：直接证明修复有效——bin sampling 的 max/min 比率从 Hilbert 的 2.40 降到 cycle-by-cycle 的 **1.01**（几乎完全均匀）。
- **V8 (双峰检测)**：增加独立的探测器，识别复杂网络下同时在波头波尾双连发的特殊耦合。
- **V9 (极端不死机)**：对于全 0、全 NaN、或不满足周期的极端垃圾电波片段，保证不崩溃退出，理智返回 `ok=False` 拒收。

修复没解决的问题（诚实说明局限）

Cycle-by-cycle 方法依赖能找到 **r_ctx** 的 **peak**。如果某个 DE candidate 产生的 **r_ctx** 是平坦的（model silent）或 peak 不够突出，函数会返回 `ok=False`。这是设计上的选择——拒绝在 ill-defined 信号上做 PAC——但意味着 DE optimizer 看到的 "feasible space" 会比 v7 略窄。

最终结论：基于 Cycle-by-cycle 的计算全面克服了 v7 阶段的心跳刺针波动陷阱，完全值得信赖用于之后的适应度打分中！

第二部分：代码逐块详解

下面把 `compute_pac_metrics_fixed.py` 拆成 6 块，逐一讲解每块在做什么、为什么这样做、用玩具数字验证。

第一块：函数入口和 sanity checks

代码

```
python
```

```
def compute_pac_metrics(r_ctx, r_thal, fs, ...):
    out = {
        "mi": 0.0,
        "preferred_phase": np.pi,          # 注意：默认是 $\pi$ 不是0
        "phase_argmax": np.pi,
        "phase_concentration": 0.0,
        "up_down_ratio": 0.0,
        "bimodality_flag": False,
        "n_so_cycles": 0,
        "ok": False,
    }

    if len(r_ctx) < int(2 * fs) or len(r_thal) < int(2 * fs):
        return out
    if r_ctx.std() < 1e-6 or r_thal.std() < 1e-6:
        return out
```

关键设计：fail-safe defaults

为什么 `preferred_phase = np.pi` 而不是 0？

如果计算失败但函数返回了 `phase = 0`，下游 fitness function 会看到"完美UP-locked！"——把 failed solution 误判成 best！

`np.pi` ($\pm 180^\circ$, DOWN trough) 是离 UP peak 最远的相位。这样：

- T10 检查 `|phase_argmax - 0| ≤ 50°` 会 fail (181° 远超 50°)
- 失败的 solution 不会被误判成好

这是 **fail-safe design**：失败不仅返回错误 flag (`ok=False`)，连数值也设成最差。即使下游忘了 check `ok`，结果也不会被误用。

Sanity check 防什么？

DE optimizer 会探索一些糟糕的参数组合——例如某些参数让模型完全 silent，`r_ctx` 全是 0。这种情况下 `find_peaks` 找不到任何 peak，后面的代码会 crash。提前 return 避免在 bad input 上浪费计算。

第二块：Cycle-by-cycle SO Phase（核心修复！）

代码

```
python
```

```

ctx_range = r_ctx.max() - r_ctx.min()
if ctx_range < 1e-6:
    return out

prominence = SO_PEAK_PROMINENCE_FRAC * ctx_range
ctx_peaks, _ = find_peaks(
    r_ctx,
    distance=int(SO_MIN_PERIOD_S * fs),
    prominence=prominence,
)

if len(ctx_peaks) < 3:
    return out

mean_isi_s = (ctx_peaks[-1] - ctx_peaks[0]) / fs / (len(ctx_peaks) - 1)
if not (0.5 <= mean_isi_s <= 4.0):
    return out

so_phase = np.full(len(r_ctx), np.nan)
for i in range(len(ctx_peaks) - 1):
    p0 = ctx_peaks[i]
    p1 = ctx_peaks[i + 1]
    cycle_len = p1 - p0
    rel = (np.arange(p0, p1) - p0) / cycle_len
    phase = np.where(rel < 0.5,
                     2 * np.pi * rel,
                     2 * np.pi * (rel - 1))
    so_phase[p0:p1] = phase

```

玩具例子

假设 `r_ctx = [0, 0, 50, 0, 0, 0, 0, 60, 0, 0, 0, 0, 55, 0]` 共14个点，每秒1点。

第1步：`find_peaks` 找出三个峰，索引在 `[2, 7, 12]`（这是Python索引，从0开始）。

第2步：检查 cycle 数 ≥ 3 ✓，平均周期 5 秒（在合理范围）✓。

第3步：构建相位。从 peak 索引 2 到 peak 索引 7 这5个samples，phase 的值如下表：

sample idx	rel = (i-p0)/cycle_len	rel<0.5?	phase 公式	phase (rad)	phase (度)
2 (peak)	0/5 = 0.0	是	$2\pi \times 0.0$	0.000	0°
3	1/5 = 0.2	是	$2\pi \times 0.2$	1.257	+72°
4	2/5 = 0.4	是	$2\pi \times 0.4$	2.513	+144°
5	3/5 = 0.6	否	$2\pi \times (0.6-1)$	-2.513	-144°
6	4/5 = 0.8	否	$2\pi \times (0.8-1)$	-1.257	-72°

关键观察：peak 上 phase = 0；上半周期 phase 从 0 上升到 + π ；下半周期 phase 从 - π 上升回 0。索引 0、1、12、13 在第一个 peak 前或最后 peak 后，没有完整周期可以插值，所以是 NaN。

为什么这样比 Hilbert 方法好？

	v7 Hilbert	修复版 cycle-by-cycle
在真实UP peak上的phase	不稳定 (-22°漂移到+8°)	永远是0° (by construction)
DOWN flat baseline 上的phase	停留在 $\pm\pi$ 附近, over-sample	按时间均匀走过, 每bin等量样本
是否依赖 Bedrosian 定理	是	否
对波形形状的鲁棒性	极差	完美

核心洞察：cycle-by-cycle 方法的相位**只依赖时间**，不依赖信号值。在 DOWN baseline 上信号是平的没关系——phase 该走还是走，按"时间走过了多少"来定义，不按"信号长啥样"来定义。这就是为什么修复完全规避了 Bedrosian 违反——因为我们**根本不用 Hilbert 变换**来求相位了。

设计要点

- **prominence 用比例 ($0.3 \times \text{ctx_range}$) 而不是绝对值：**让阈值自适应不同 DE candidate 的信号幅度。
- **distance=0.7s：**防止把小波动当成 peak，对应 SO 最高 1.4 Hz。
- **mean_isi 检查：**拒绝 SO 频率不合理的情况 (< 0.25 Hz 或 > 2 Hz)。
- **NaN 填充：**peak 之前和最后 peak 之后的 samples 没有完整周期可插值，标 NaN 让下游跳过。

第三块：Spindle Envelope（这块没改）

代码

```
python
```

```
sos_sp = butter(4, [SPINDLE_LO, SPINDLE_HI],
                btype="band", fs=fs, output="sos")
sp_filt = sosfiltfilt(sos_sp, r_thal)
sp_amp = np.abs(hilbert(sp_filt))
```

三步流程

步骤	公式	物理含义
Bandpass	<code>sosfiltfilt(sos, r_thal)</code>	只保留 10-14 Hz 的 spindle 振荡，去除 baseline 和噪声
Hilbert	<code>hilbert(sp_filt)</code>	把振荡信号变成"复信号"，包含瞬时振幅+相位
取绝对值	<code>np.abs(...)</code>	取振幅，得到 envelope（spindle 强度的"轮廓线"）

为什么这块不需要修复？

回想 v7 的 bug 出在 SO 的 phase（Bedrosian 违反）。但 spindle 的 envelope 没有 bug：

	Phase (<code>np.angle</code>)	Envelope (<code>np.abs</code>)
v7 是否有 bug	是	否
需要 Bedrosian 定理	是	否
输入要求	必须是 cosine-like	任何窄带信号都行

简单说：取 angle 需要信号是 cosine-like，但取 absolute value 不需要。Spindle band 里的信号本身就是窄带的振荡，所以 envelope 一直是有意义的。

注意：envelope 永远 ≥ 0

`sp_amp` 是绝对值，永远是非负。如果有人误用了错误的 spindle band（比如 [50, 100] Hz），spindle 信号会被滤掉，envelope 会变成"接近0的小非负值"——不是震荡。

第四块：Edge Trimming

代码

```
python
```

```
edge = int(0.5 * fs)
so_phase = so_phase[edge:-edge]
sp_amp = sp_amp[edge:-edge]
valid = ~np.isnan(so_phase)
if valid.sum() < int(5 * fs):
    return out
so_phase_v = so_phase[valid]
sp_amp_v = sp_amp[valid]
```

为什么要去边缘？

`sosfiltfilt`（双向带通滤波）在信号开头和结尾会产生**瞬态 artifact**——滤波器需要"看到一段历史"才能稳定输出。在信号开头那一瞬间，滤波器还没"热身"，输出值不可信。

实测数值

对一个恒定振幅 30 的纯 13Hz 信号做滤波：

- t=0.00s（最边缘）：sp_amp = 12（错的，应该是30）
- t=0.10s（边缘区）：sp_amp = 25（仍偏低）
- **t=0.50s（trim后）：sp_amp = 30 ✓**
- t=4.00s（中间）：sp_amp = 30 ✓
- t=7.90s（接近尾边缘）：sp_amp = 3（artifact）

工程妥协

理论上 SO bandpass 的 1% impulse response 宽度是 5.3 秒，应该 trim 2.65 秒每边。但代码只 trim 0.5 秒——

edge值	损失数据	artifact残留	优劣
0.0秒 (不trim)	0%	严重artifact	X 数据被污染
0.5秒 (现行)	1.7%	小artifact残留	✓ 现实选择
2.65秒 (理论彻底)	8.8%	几乎无artifact	X 损失太多

代码作者的判断：**0.5秒足够好**——清掉了大部分 artifact，又保留了大部分数据。这不是 bug，是工程权衡。

`valid` 检查

```
python
```



```
valid = ~np.isnan(so_phase)
if valid.sum() < int(5 * fs):
    return out
```

记得 Step 1 末尾的 NaN 吗？第一个 peak 之前 + 最后 peak 之后 + 整个边缘 trim 区，都是 NaN。这一步：

1. 排除 NaN
2. 检查剩下有效 samples ≥ 5 秒
3. 取出有效部分给后面 MI 计算用

会触发失败的场景：r_ctx peaks 太稀疏（比如60秒只有3个peak挤在最后10秒）→ 大量 NaN → valid < 5秒 → 直接 return out。

第五块：Tort 2010 KL Modulation Index

代码

```
python

bin_edges = np.linspace(-np.pi, np.pi, PAC_N_BINS + 1)
bin_centers = (bin_edges[:-1] + bin_edges[1:]) / 2
mean_amp = np.zeros(PAC_N_BINS)
for i in range(PAC_N_BINS):
    mask = (so_phase_v >= bin_edges[i]) & (so_phase_v < bin_edges[i + 1])
    if mask.any():
        mean_amp[i] = sp_amp_v[mask].mean()

total = mean_amp.sum()
p = mean_amp / total
p_safe = np.where(p > 0, p, 1.0)
H = -np.sum(p * np.log(p_safe))
mi = (np.log(PAC_N_BINS) - H) / np.log(PAC_N_BINS)
mi = float(np.clip(mi, 0.0, 1.0))
```

玩具计算（4-bin 简化版）

想象SO周期被切成4个相位箱：

bin 0	bin 1	bin 2	bin 3
$[-180^\circ, -90^\circ)$	$[-90^\circ, 0^\circ)$	$[0^\circ, +90^\circ)$	$[+90^\circ, +180^\circ]$
DOWN trough	DOWN→UP rise	UP peak	UP→DOWN fall

场景A：完美 UP-locked, spindle 集中在 UP peak

```
python
```

```
mean_amp = [1, 1, 10, 1]
```

```
total = 13
```

```
p = [1/13, 1/13, 10/13, 1/13] ≈ [0.077, 0.077, 0.769, 0.077]
```

计算 **Shannon entropy** :

```
H = -Σ p[i] · ln(p[i])  
  = -(0.077·ln(0.077)·3 + 0.769·ln(0.769))  
  = -(0.077·(-2.564)·3 + 0.769·(-0.262))  
  = 0.793
```

计算 **MI** :

```
MI = (ln(N) - H) / ln(N)  
    = (ln(4) - 0.793) / ln(4)  
    = (1.386 - 0.793) / 1.386  
    ≈ 0.428
```

MI 直觉表 (4-bin)

分布	描述	H	MI
[5, 5, 5, 5]	完全均匀	1.386	0.000
[3, 5, 7, 5]	弱偏向UP	1.358	0.030
[2, 4, 8, 6]	中等偏UP	1.301	0.077
[1, 1, 10, 1]	显著UP-locked	0.793	0.428
[0, 0, 50, 0]	完美locked	0	1.000

三个事实

事实1 : **MI** = "归一化的非均匀性"

$$MI = \frac{\ln(N) - H}{\ln(N)}$$

- 分子：你的分布**比均匀分布**少多少分散性
- 除以 $\ln(N)$ ：把数值压到 [0,1]
- 完全均匀 $\rightarrow MI = 0$ ；完全集中 $\rightarrow MI = 1$

事实2：除以 $\ln(N)$ 是为了可比性

不同 bin 数时 $\ln(N)$ 不同 ($N=4 \rightarrow 1.386$; $N=18 \rightarrow 2.890$)。除以 $\ln(N)$ 让 MI 永远在 $[0, 1]$ ，可以跨不同 bin 数比较。

事实3：为什么有 `p_safe` ？

数学上 $0 \times \ln(0)$ 是不定式。但如果强制把 $\ln(0)$ 替换成 $\ln(1) = 0$ ，乘以前面的 0 还是 0。所以 `p_safe = np.where(p > 0, p, 1.0)` 是个数值技巧，等价于 " $0 \times \log(\text{任何东西}) = 0$ "。

V7 的 MI = 0.012 是什么水平？

类比 4-bin 表里 `[2, 4, 8, 6]` (moderate UP-leaning)。文献 Helfrich 2018 healthy young adult 典型 MI $\approx 0.02-0.08$ 。V7 落在弱耦合区间。

第六块：三个 Phase Indicators + Bimodality

代码

```
python

# (A) Histogram-based MVL
mv1_hist = (p * np.exp(1j * bin_centers)).sum()
preferred_phase = float(np.angle(mv1_hist))
phase_concentration = float(np.abs(mv1_hist))

# (B) Argmax bin
phase_argmax = float(bin_centers[int(np.argmax(mean_amp))])

# (C) UP/DOWN ratio
up_mask = np.abs(bin_centers) <= (np.pi / 2)
up_w = float(mean_amp[up_mask].sum())
down_w = float(mean_amp[~up_mask].sum())
up_down_ratio = up_w / max(down_w, 1e-12)

# Bimodality detection
bimodality_flag = _detect_bimodality(mean_amp, bin_centers)
```

为什么需要三个 indicator 而不是一个？

核心问题：MI 告诉你"有多强"，不告诉你"在哪"。

下面三个分布的 MI 都很高 (≈ 0.43)，但形状完全不同：

- A: [1, 1, 10, 1]

- 主峰在UP peak
- B: [10, 1, 1, 1]

- 主峰在DOWN trough
- C: [5, 1, 5, 1]

- 双峰: UP和DOWN各一个

光看 MI 你不知道是 A、B 还是 C。所以需要 **phase indicator** 告诉你 "where"——但单一 indicator 在某些分布上会 misleading。

关键陷阱：MVL 在双峰下失灵

考虑 `mean_amp = [10, 1, 10, 1]` (UP和DOWN两个峰一样高)：

Indicator	值	含义
preferred_phase (MVL angle)	-45°	错误！这个相位 amplitude 只有 1
phase_argmax	-135°	其中一个真峰
phase_concentration	0.00	警报！MVL 长度几乎为0
up_down_ratio	1.00	UP 和 DOWN 一样多

MVL 失灵的几何原因：在复平面上，DOWN 半边的箭头（指向 -135°）和 UP 半边的箭头（指向 +45°）**完全相反**——它们互相抵消，总和（MVL）几乎为0。`phase_concentration = 0` 就是告诉你 "MVL 不可信" 的警报。

三个 indicator 的分工

Indicator	何时最有用	何时会失灵	鲁棒性
preferred_phase	单峰分布	双峰 → 抵消产生伪角度	低
phase_argmax	任何分布	离散精度只有20°	高
up_down_ratio	任何分布	损失精度细节	极高
phase_concentration	辅助警报，告诉你前面那些可信不可信	—	—

实际使用策略：

- 如果 `bimodality_flag=False` 且 `phase_concentration > 0.1` → 信任 `preferred_phase`
- 如果 `bimodality_flag=True` 或 `phase_concentration` 低 → 用 `phase_argmax` + `up_down_ratio`

详解 up_down_ratio

这个指标同时关心两个信号：

- **SO 相位**决定每个 bin 属于哪半（UP半 or DOWN半）
- **Spindle 振幅**决定每个 bin 的"高度"

18-bin 的 UP 和 DOWN 半边划分（注意 $\pm 90^\circ$ 边界）：

Bin	相位中心	$\text{abs(相位)} \leq 90^\circ?$	哪半边
0~3	$-170^\circ \sim -110^\circ$	否	DOWN
4	-90°	是 ($\leq 90^\circ$)	UP
5~13	$-70^\circ \sim +90^\circ$	是	UP
14~17	$+110^\circ \sim +170^\circ$	否	DOWN

⚠ **重要：** $\leq 90^\circ$ 让边界点 $\pm 90^\circ$ 算到 UP 半。所以 UP 半是 **10个 bin**，DOWN 半是 **8个 bin**（不是 9对9）。这意味着**uniform 分布的 up_down_ratio 不是 1，而是 $10/8 = 1.25$ ！**

up_down_ratio 三道练习

题1： $\text{mean_amp} = [10, 10, \dots, 10]$ (全是10) $\rightarrow$ ratio = ?

UP半总和 = $10 \times 10 = 100$
DOWN半总和 = $10 \times 8 = 80$
ratio = $100 / 80 = 1.25$ ← uniform 的 baseline

题2： $\text{mean_amp} = [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 20, 20, 20, 20, 20, 2, 2, 2, 2]$ （前9个 bin是2，bin 9~13是20，最后4个bin是2）

UP半 (bin 4~13): bin 4~8 是 2 $\rightarrow 5 \times 2 = 10$; bin 9~13 是 20 $\rightarrow 5 \times 20 = 100$
UP半总和 = $10 + 100 = 110$
DOWN半 (bin 0~3 + 14~17): 全是 2 $\rightarrow 8 \times 2 = 16$
ratio = $110 / 16 \approx 6.88$ ← 强烈 UP-locked

题3： $\text{up_down_ratio} = 0.3$ 物理含义？

spindle 主要在 SO 的 DOWN 半边爆发，UP 半的能量只有 DOWN 半的 30%——这是 **DOWN-locked** 模式。

up_down_ratio 解读速查

ratio 范围	含义
≈ 1.25	uniform 分布（基线）
> 1.25 显著	spindle UP-locked（健康）

测量项	V7 Hilbert	Fixed cycle-by-cycle
Bin sampling max/min ratio	2.40（不均匀）	1.01（几乎完美）
在合成 UP-locked 数据上 recovered phase	-12.2°（biased）	-0.0°（正确）
对波形形状的鲁棒性	极差（漂移随波形变化）	完美（4种波形都给出+10°）
是否依赖 Bedrosian 定理	是	否

V7 best solution 在新指标下的真实样子

指标	值	解读
MI	0.012	弱-中等耦合（落在 Helfrich healthy 范围下沿）
phase_argmax	+10°	接近 UP peak （与 v7 报告的 +159° 完全相反！）
phase_concentration	0.087	低（中老年范围）
up_down_ratio	1.46	UP-locked（>1.25 baseline）
bimodality_flag	True	双峰耦合

关键反转：V7 模型其实是 UP-locked 的弱-中耦合，之前报的 +159° 是 Hilbert phase 计算错误——不是模型生理学有问题，是计算方法有问题。

第四部分：用一句话快速讲明白整个修复

"PAC 算法本来想测'spindle 最爱在 SO 的哪个相位爆发'。
老方法用滤波器 + Hilbert 变换算 SO 相位 — 这套方法的前提是 SO 是平滑正弦波。
但我们模型产出的 SO 是**尖锐脉冲 + 长平地**，不是正弦波。
滤波器把脉冲'抹平'后，相位定义比真实 UP peak晚了 50ms，DOWN 平地又**采样过密**。
结果耦合相位被错算成 +159°（DOWN trough 附近）而不是真实的 0°（UP peak）。
修复方法：跳过滤波器，直接在原始信号上找 peak，**peak 所在时刻就是相位 0°**。
用 9 个独立 test 验证（包括各种波形、噪声、bimodal、edge case）— 全部通过。"

短、准、有数据 — 适合答辩或组会快速过一遍。